

The 40th IEEE/ACM International Conference
on Automated Software Engineering (ASE 2025)



北京科技大学
University of Science and Technology Beijing



DIFFFIX: Incrementally Fixing AST Diffs via Context and Type Information

Guofeng Zeng

d202110394@xs.ustb.edu.cn

Chang-ai Sun

casun@ustb.edu.cn

Kai Gao

kai.gao@ustb.edu.cn

Huai Liu

hliu@swin.edu.au

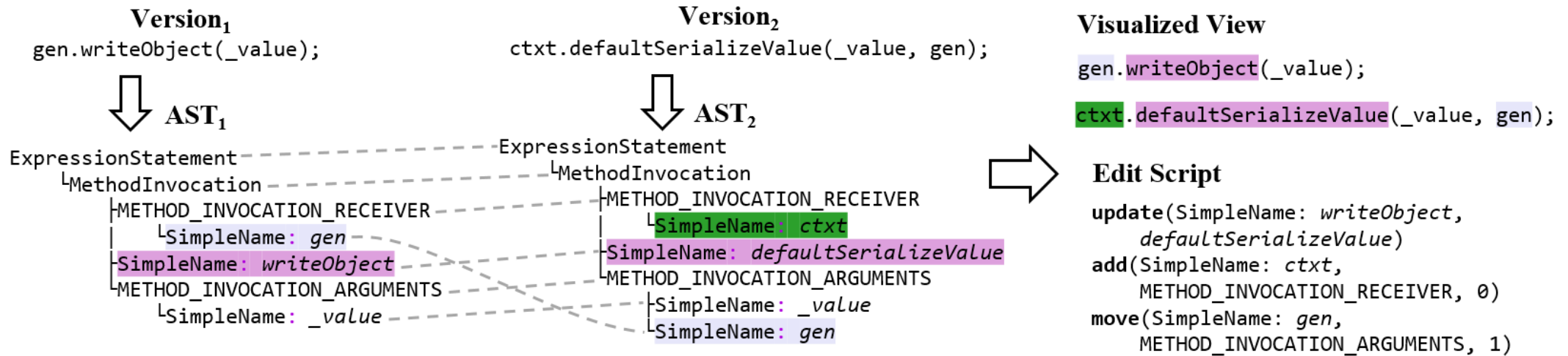
Seoul, November 19, 2025

Overview of AST Differencing (ASTDiff)

■ Process

- (1) parse two versions of programs into ASTs
- (2) compare ASTs to get the diff (node mappings)
- (3) generate edit script / visualize syntactic change

A fundamental approach to capture structural and syntactic changes of program elements



■ Techniques

ChangeDistiller, GumTree, MTDiff, IJM, iASTMapper, RM-ASTDiff, etc.

■ Applications

automated program repair, change pattern mining, refactoring detection, etc.

Accuracy Issue of ASTDiff Techniques

■ Can the generated diffs accurately reflect the code changes?

■ Limitations of existing ASTDiff techniques

- Unsound node matching heuristic
- Insufficient use of program information
- Trade-off between accuracy and efficiency
- Mutual interference of inaccuracies

} Struggle to generate accurate diffs, resulting in **20%-40% inaccuracies**^[1,2]

Motivation:

Since inaccurate mappings are difficult to prevent, can we identify and fix them in the generated diff to improve the accuracy?

[1] Y. Fan, X. Xia, D. Lo, et al. A differential testing approach for evaluating abstract syntax tree mapping algorithms. ICSE, 2021.

[2] P. Alikhanifard and N. Tsantalis. A novel refactoring and semantic aware abstract syntax tree differencing tool and a benchmark for evaluating the accuracy of diff tools. TOSEM, 2025.

Investigation of Inaccurate Mapping Characteristics

■ Dataset: DiffBenchmark

- Human-verified oracle diffs for 987 commits
 - 800 bug-fixing commits from Defects4J
 - 187 refactoring commits from RefOracle

■ Subjects: 5 State-of-the-art ASTDiff techniques

- RM-ASTDiff (RMD)
- iASTMapper (IAM)
- DiffAutoTuning (DAT)
- GumTree-simple (GTS)
- GumTree (GT)

■ Three Inaccuracy Types

- Missing, Wrong, Arbitrary

Missing Mapping

MT2: generated by IAM for Compress/32

```
currEntry.setGroupId(Integer.parseInt(val));
```

```
currEntry.setGroupId(Long.parseLong(val));
```

Wrong Mapping

WB1: generated by DAT>S> for zeromq/jeromq/02d3fa1

```
public void close() { hunk1 X.. }
```

```
public void close() { ... try { hunk1 } ... }
```

Arbitrary Mapping

AE1: generated by RMD>S for Closure/122

```
if (comment.getValue().indexOf("/* @") != -1) { ... }
```

```
if (p.matcher(comment.getValue()).find()) { ... }
```

Observation #1: Context-related

- Inaccurate mappings commonly reside in four contexts of mapped nodes

Parent Context

inaccuracies under mapped parents

WB₁: generated by DAT>S> for zeromq/jeromq/02d3fa1

```
public void close() { hunk1 ... }
```

```
public void close() { ... try { hunk1 } ... }
```

Children Context

inaccuracies upon mapped children

WE2: generated by RMD&DAT for Compress/46

```
if (1 >= TWO_TO_32) { ... }
```

```
if (1 < Integer.MIN_VALUE || 1 > Integer.MAX_VALUE) { ... }
```

Inner Context

inaccuracies inner mapped blocks

WE1: generated by RMD for Compress/2

```
if (offset % 2 != 0) { read(); } X
```

```
if (currentEntry != null) { ... read() ... }
```

```
if (offset % 2 != 0) { if (read() < 0) { return null; } }
```

Nearby Context

inaccuracies around mapped statements

WS1: generated by RMD for JacksonCore/10

```
hash ^= (hash >>> 12);
```

```
hash ^= (hash << 3); hash += (hash >>> 12);
```

Implication:

Context can be used to locate potentially inaccurate mappings.

Observation #2: Type-related

- The syntax types of nodes constraint their mappings

Roles in Declaration

MT1: generated by GT for CyanogenMod/android_frameworks_base/658a918

```
public class PhoneStatusBar extends BaseStatusBar ... { ...  
protected PhoneStatusBarView makeStatusBarView() { ... }  
public class PhoneStatusBar extends BaseStatusBar ... { ...  
protected PhoneStatusBarView makeStatusBarView() { ... }  
}
```

Infix Expression Compatibility

WE2: generated by RMD&DAT for Compress/46

```
if (1 >= TWO_TO_32) { ... }  
if (1 < Integer.MIN_VALUE || 1 > Integer.MAX_VALUE) { ... }
```

Block Ownership

WB1: generated by DAT>S> for zeromq/jeromq/02d3fa1

```
public void close() { hunk1 ... }  
public void close() { ... try { hunk1 } ... }
```

- Certain types of nodes are prone to inaccurate mappings

Simple Name (Variable)

AT1: generated by RMD for infinispn/infinispn/03573a6

```
boolean isPre = eventImpl.isPre();  
if (!observation.shouldInvoke(isPre)) return null;  
protected boolean shouldInvoke(Event<K, V> event) {  
return observation.shouldInvoke(event.isPre());  
... if (!shouldInvoke(event)) return null;  
}
```

Method Invocation

AE1: generated by RMD>S for Closure/122

```
if (comment.getValue().indexOf("/") != -1) { ... }  
if (p.matcher(comment.getValue()).find()) { ... }
```

Statement

MS1: generated by GTS for hierynomus/sshj/7c26ac6

```
{ ... sshClient.connect("172.16.37.129"); ... }  
{ ... sshClient.connect(server.getHost(),  
server.getPort()); ... }
```

Implication:

Type-specific constraints should be utilized to identify and fix inaccurate mappings, especially for certain types of nodes.

Observation #3: Coexisting

- Different types of inaccurate mappings **coexist** in the generated diff, reflecting the **mutual interference** of inaccuracies

ME₁: generated by RMD for Cli/37

```
return token.startsWith("-") && token.length() >= 2
&& options.hasShortOption(token.substring(1, 2));

if (!token.startsWith("-") || token.length() == 1)
{ return false; }
int pos = token.indexOf("=");
String optName = pos == -1 ?
    token.substring(1) : token.substring(1, pos);
return options.hasShortOption(optName);
```

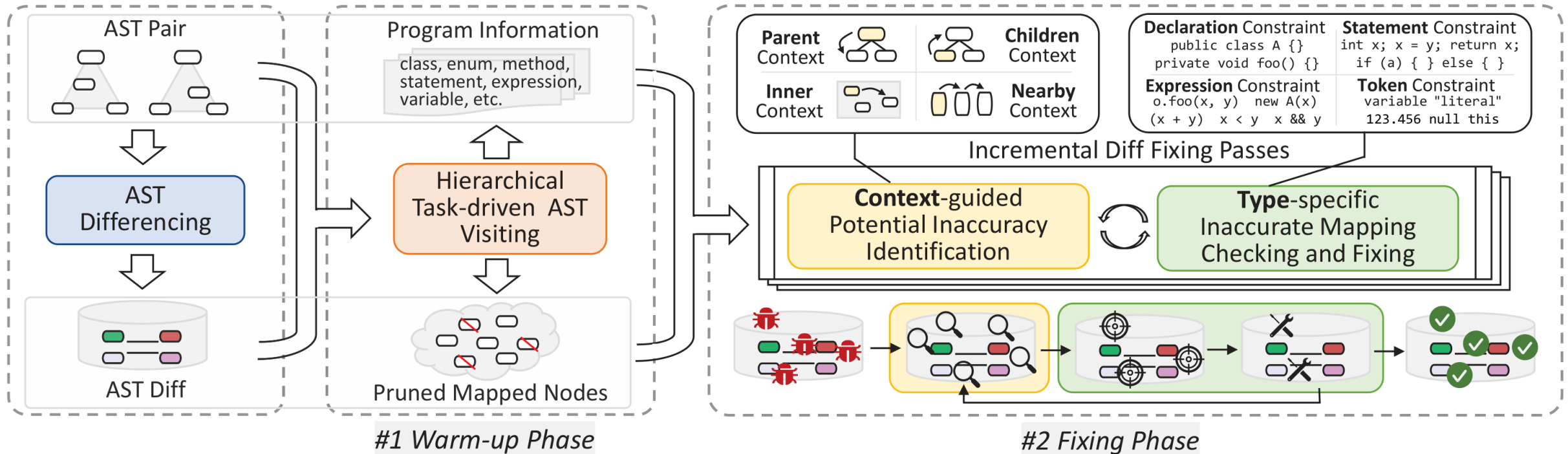
Implication:

Coexisting inaccurate mappings should be identified iteratively and incrementally.

WS&MS: generated by RMD for Time/23

map.put("GMT", "UTC"); ...	map.put("GMT", "UTC");
map.put("WET", "Europe/London");	map.put("WET", "WET");
map.put("ECT", "Europe/Paris");	map.put("CET", "CET");
map.put("ART", "Africa/Cairo");	map.put("MET", "CET");
map.put("CAT", "Africa/Harare");	map.put("ECT", "CET");
map.put("EET", "Europe/Bucharest");	map.put("EET", "EET"); ...
map.put("EAT", "Africa/Addis_Ababa");	map.put("ART", "Africa/Cairo");
map.put("MET", "Asia/Tehran");	map.put("CAT", "Africa/Harare");
map.put("IST", "Asia/Calcutta");	map.put("EAT", "Africa/Addis_Ababa"); ...
map.put("BST", "Asia/Dhaka");	map.put("IST", "Asia/Kolkata");
map.put("VST", "Asia/Saigon");	map.put("BST", "Asia/Dhaka");
	map.put("VST", "Asia/Ho Chi Minh");

Our Approach: DIFFFIX



■ **Input:** A pair of ASTs and their diff generated by an ASTDiff technique

■ **Output:** A refined diff with inaccurate mappings fixed

#1 Warm-up Phase performs initializations and optimizations

#2 Fixing Phase performs identification and fixing of inaccuracies

Warm-up Phase

■ Three tasks to establish the initial environment

(1) *node type checking*

whether two mapped nodes have the same types

(2) *identical mapping checking*

whether subtrees are identical and consistently mapped

(3) *global information collection*

maintains info that across the whole AST

■ Three tasks to optimize the fixing phase

(4) *local information collection* (opt_1)

maintains shared type-specific info

(5) *declaration-level pruning* (opt_2)

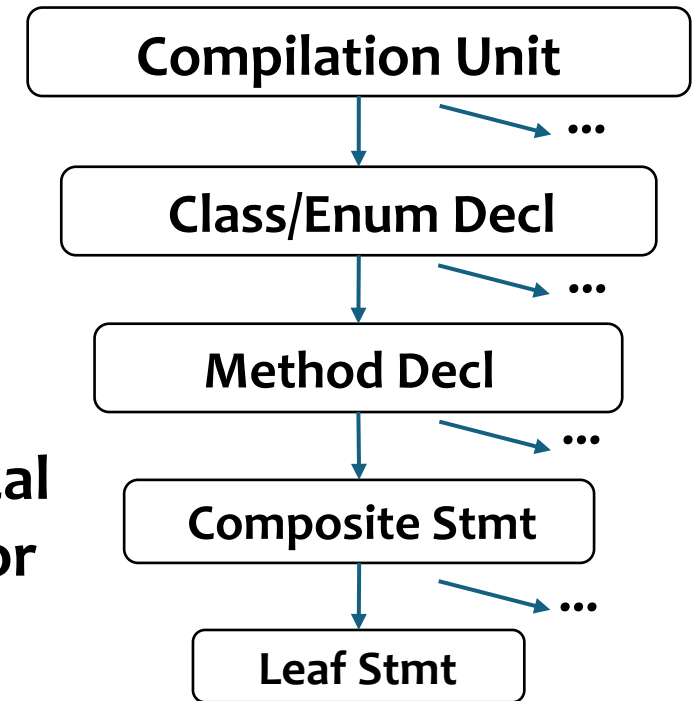
filter descendants of identical declarations

(6) *statement-level pruning* (opt_3)

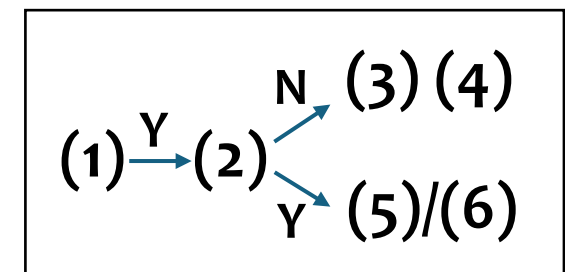
filter descendants of identical statements

(two pruned node set, one remains identical stmt roots)

Hierarchical
AST Visitor

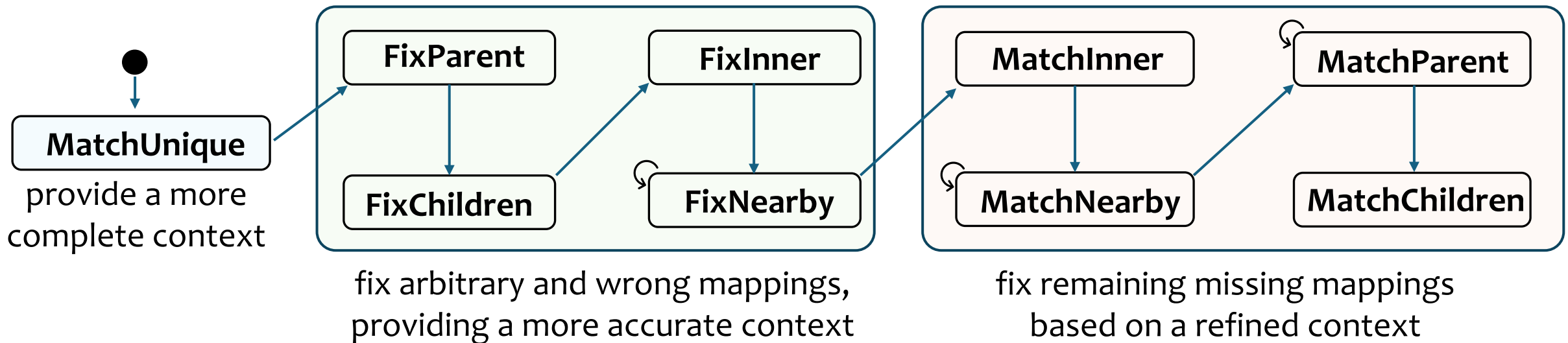
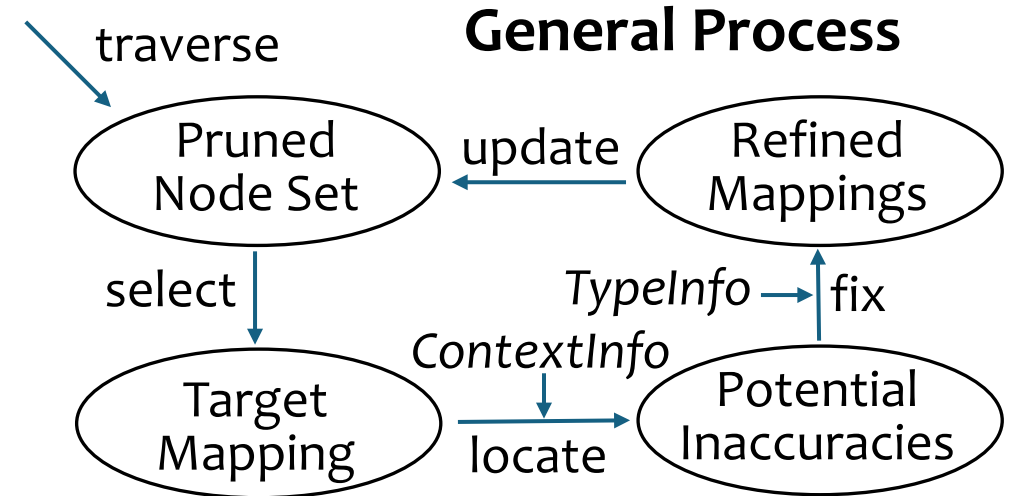


Task-driven
Manner



Fixing Phase: 9 passes

- Each fixing pass traverses one of two pruned node sets, leverages a specific context to **locate potential inaccuracies around a target mapping**, and **uses both general and type-specific checks and operations** to confirm and fix them
- **The order of fixing passes** is arranged according to their **interdependencies**



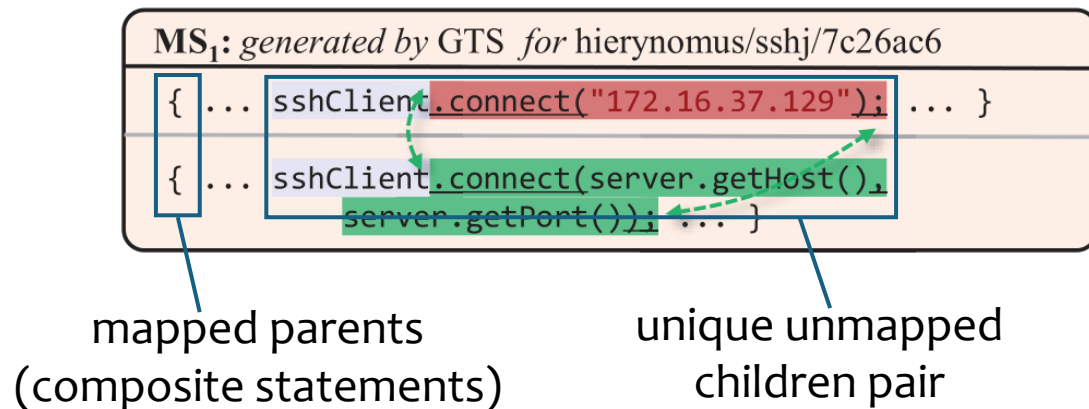
Execution Flow of Fixing Passes

Fixing Pass: MatchUnique

- **Observation:** For two mapped (a) composite statements or (b) class declarations, if each of them contains a **unique unmapped child node of a specific type**, then the pair of the two child nodes are **likely to be a missing mapping**.

- **Process**

- (1) traverse the pruned node set to identify such the unique unmapped pair
- (2) check whether the pair of nodes have common elements
- (3) use a refined subtree matching algorithm to fix missing mappings



Algorithm 1: MatchUnique

Data: Pruned mapped AST_1 nodes $\mathcal{P}$

```
1 foreach  $m_1 \in \mathcal{P}$  do
2   if isMapped( $m_1$ ) then
3      $m_2 \leftarrow getMapped(m_1)$ 
4     if isCompositeStatement( $m_1$ ) then
5        $\mathcal{U} \leftarrow uniqueUnmappedChildrenPairs(m_1, m_2)$ 
6       foreach  $(u_1, u_2) \in \mathcal{U}$  do
7         | tryToMatchInComposite( $u_1, u_2$ ) (a)
8       end
9     end
10    else if isClassDeclaration( $m_1$ ) then
11       $\mathcal{D} \leftarrow uniqueUnmappedChildrenPairs(m_1, m_2)$ 
12      foreach  $(d_1, d_2) \in \mathcal{D}$  do
13        | tryToMatchInClass( $d_1, d_2$ ) (b)
14      end
15    end
16  end
17 end
```

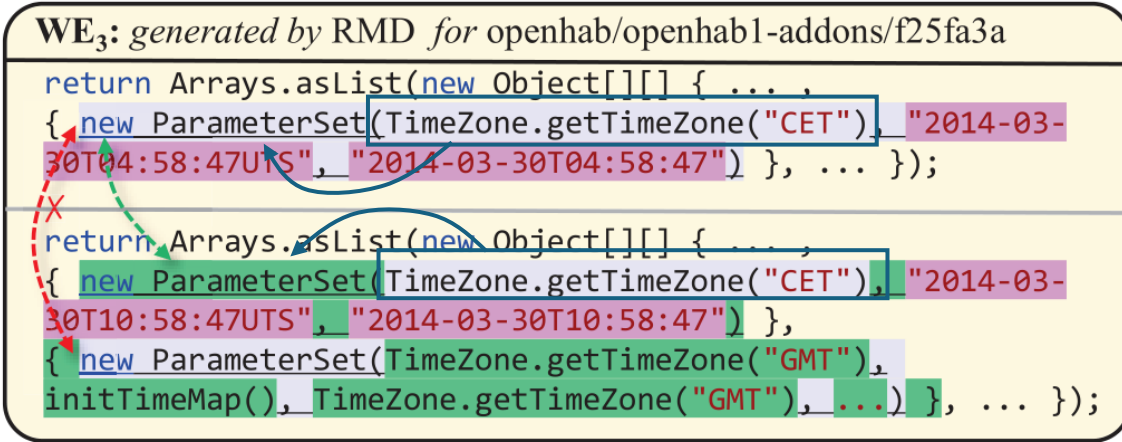
Fixing Pass: FixParent, FixChildren, and FixInner

- **FixParent** identifies **suboptimal mappings of parents** that have mapped child nodes

WE₃: generated by RMD for openhab/openhab1-addons/f25fa3a

```
return Arrays.asList(new Object[][] { ... ,
{ new ParameterSet(TimeZone.getTimeZone("CET"), "2014-03-30T04:58:47UTC", "2014-03-30T04:58:47") }, ... });
```

```
return Arrays.asList(new Object[][] { ... ,
{ new ParameterSet(TimeZone.getTimeZone("CET"), "2014-03-30T10:58:47UTC", "2014-03-30T10:58:47") },
{ new ParameterSet(TimeZone.getTimeZone("GMT"), initTimeMap(), TimeZone.getTimeZone("GMT"), ...) }, ... });
```

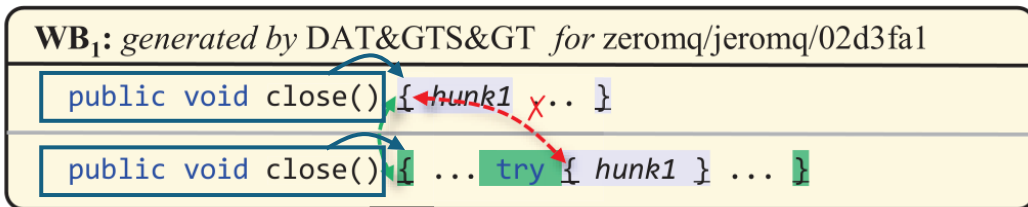


- **FixChildren** identifies **mis-mapped child nodes** that have an unique role

WB₁: generated by DAT>S> for zeromq/jeromq/02d3fa1

```
public void close() { hunk1 ... }
```

```
public void close() { ... try { hunk1 } ... }
```



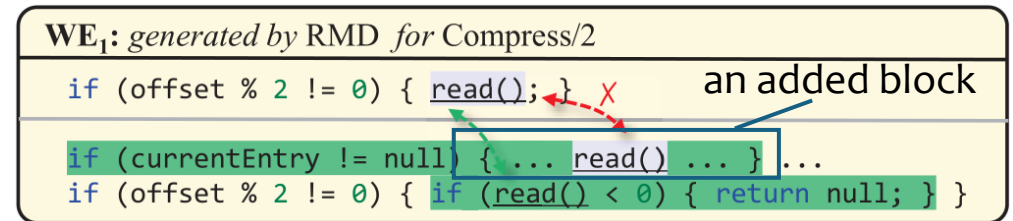
- **FixInner** identifies **arbitrary mappings**
(1) exist solely in added/deleted **blocks**
or
(2) are incompatible method invocations

WE₁: generated by RMD for Compress/2

```
if (offset % 2 != 0) { read(); } X an added block
```

```
if (currentEntry != null) { ... read() ... } ...
```

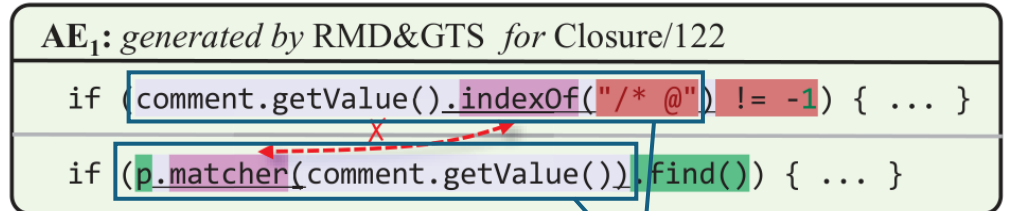
```
if (offset % 2 != 0) { if (read() < 0) { return null; } }
```



AE₁: generated by RMD>S for Closure/122

```
if (comment.getValue().indexOf("/* @") != -1) { ... }
```

```
if (p.matcher(comment.getValue()).find()) { ... }
```



incompatible with return types

Fixing Pass: FixNearby

- Iteratively checks 4 types of **inaccurate mappings in nearby (previous and next) statements**

- (1) suboptimal mappings with better nearby partners
- (2) mis-mapped nearby statements
- (3) mis-mapped identical statements
- (4) mis-mapped moved nodes in nearby statements

WS1: generated by RMD for JacksonCore/10

```
hash ^= (hash >>> 12);
hash ^= (hash << 3); hash += (hash >>> 12);
```

Annotations: "better mapped elements" points to the second hash shift; "next stmt" points to the second statement.

WS2: generated by RMD for Closure/25

```
Node constructor = n.getFirstChild();
scope = traverse(constructor, scope);
... for (...) { scope = traverse(arg, scope); }
scope = traverseChildren(n, scope);
Node constructor = n.getFirstChild();
```

Annotations: "next stmt" points to the second statement; "prev stmt" points to the first statement.

WS3: generated by RMD for Closure/75

```
switch(c){case '\u000B': return TernaryValue.TRUE;
... case '\uFEFF': return TernaryValue.TRUE; ... }
switch(c){case '\u000B': return TernaryValue.UNKNOWN;
... case '\uFEFF': return TernaryValue.TRUE; ... }
```

Annotation: "same position" points to the same case in both switches.

WT₁: generated by IAM&DAT>S> for Closure/97

```
result = lvalInt >>> rvalInt;
long lvalLong = lvalInt & 0xffffffffL;
result = lvalLong >>> rvalInt;
```

Annotations: "X" marks the shift of lvalInt in the first and third lines.

Algorithm 2: FixNearby

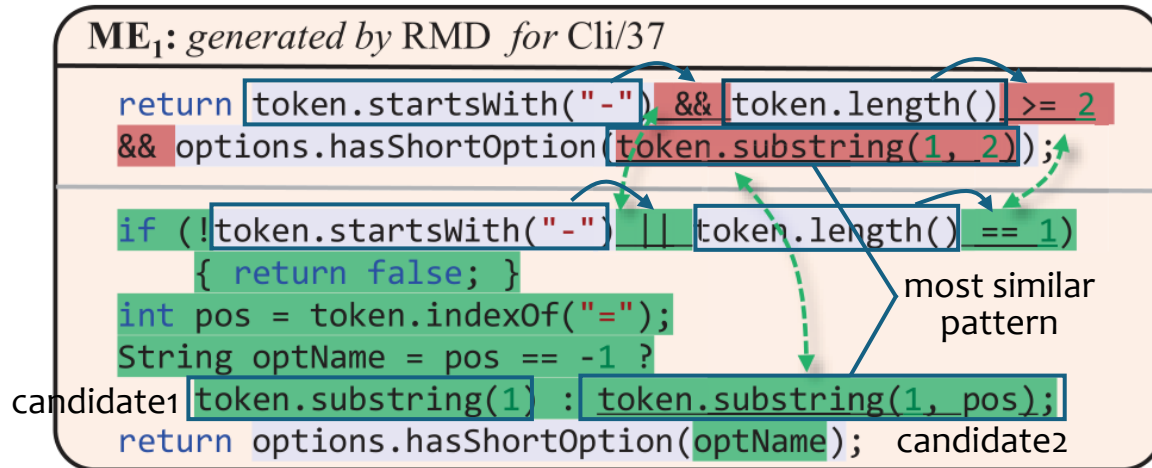
Data: Pruned mapped AST_1 nodes $\mathcal{P}'$

```
1 Function FixNearbyRound( $\mathcal{S}$ ):
2    $\mathcal{R} \leftarrow \emptyset$ 
3   foreach  $s_1 \in \mathcal{S}$  do
4     if  $isMapped(s_1)$  then
5       (1)  $s_2 \leftarrow getMapped(s_1)$ 
6       if  $checkNearbyBetter(s_1, s_2, \mathcal{R})$  then
7         continue
8       (2) end
9       (3)  $checkNearbyWM(s_1, s_2, \mathcal{R})$ 
10       $checkIdenticalWM(s_1, s_2, \mathcal{R})$ 
11      (4)  $checkNearbyInnerStmtWM(s_1, s_2)$ 
12    end
13  end
14  return  $\mathcal{R}$ 
15   $\mathcal{S} \leftarrow getAllLeafStatements(\mathcal{P}')$ 
16  while  $\mathcal{S} \neq \emptyset$  do
17     $\mathcal{S} \leftarrow FixNearbyRound(\mathcal{S})$ 
18  end
```

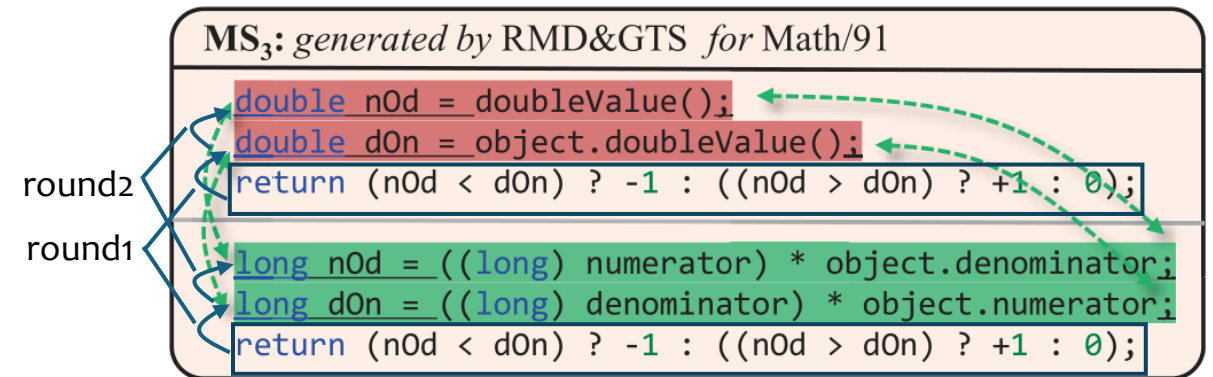
iteratively execute
checks for newly
mapped statements

Fixing Pass: The Last Four Passes

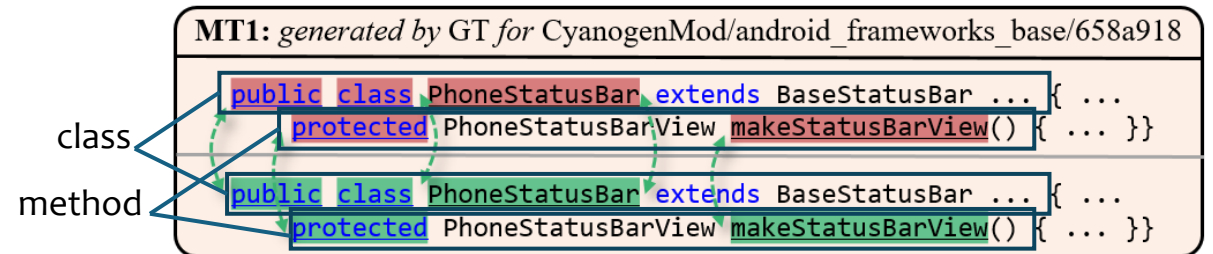
- **MatchInner** identifies **missing mappings within blocks**; uses an algorithm to find best candidates of method invocations
- **MatchParent** iteratively uses mapped nodes as anchors to identify their **missing-mapped parent nodes**



- **MatchNearby** iteratively identifies **missing-mapped nearby statements** with common elements



- **MatchChildren** identifies **missing-mapped children** by parent types and their roles



Evaluation

■ Research Questions

- **RQ1 (Effectiveness):** Can DIFFFIX effectively improve **node mapping accuracy**?
- **RQ2 (Efficiency):** What is the **time cost** of DIFFFIX? Can optimizations improve its efficiency?
- **RQ3 (Effect on ES Size):** How does DIFFFIX affect **edit script size**?
- **RQ4 (Impact of Subprocesses):** What's the contributions and costs of its **subprocesses**?

■ Subjects: 5 state-of-the-art ASTDiff techniques

- RMD, IAM, DAT, GTS, GT

■ Dataset

- (1) DiffBenchmark: human-verified oracle diffs for 987 commits (for RQ1-4)
- (2) Manually crafted 10,607 diffs from commits in 116 popular GitHub projects (for RQ1)

Effectiveness (RQ1)

■ Results on DiffBenchmark (800 Defects4j commits)

- *Metrics:* PDR (Perfect Diff Rate) *perfect diff:* a diff without inaccurate mappings
- *Summary:* (1) DIFFFIX improved PDR by 5.25%-51.12% on the ASTDiff techniques
- (2) DIFFFIX applied to RMD achieved the best PDR of 95.38%

■ Results on 10,607 diffs (generated by RMD)

- DIFFFIX modified 1,688 (15.91%) diffs from commits in 115 projects
- Randomly selected 320 of the 1,688 diffs to manually verify fixing accuracy
- *Summary:* DIFFFIX accurately fixed 298 of the 320 diffs, achieved a precision of 93.12%

	PDR_{stmt} (%)	PDR_{token} (%)	Missing Mappings (#)	Arbitrary Mappings (#)	Wrong Mappings (#)
RMD	89.38	86.00	719	81	251
RMD+DIFFFIX	97.62 (+8.25)	95.38 (+9.38)	164 (-555)	81 (+0)	42 (-209)
IAM	90.38	71.00	570	384	408
IAM+DIFFFIX	92.25 (+1.88)	80.75 (+9.75)	353 (-217)	400 (+16)	372 (-36)
DAT	82.00	70.62	177	1,279	986
DAT+DIFFFIX	86.50 (+4.50)	75.88 (+5.25)	174 (-3)	1,162 (-117)	787 (-199)
GTS	72.25	62.62	1,424	356	1,006
GTS+DIFFFIX	82.88 (+10.62)	72.25 (+9.62)	759 (-665)	362 (+6)	801 (-205)
GT	75.25	17.25	3,374	717	1,035
GT+DIFFFIX	82.88 (+7.62)	68.38 (+51.12)	892 (-2,482)	637 (-80)	831 (-204)

PDR_{stmt} : only stmt mappings PDR_{token} : check all mappings

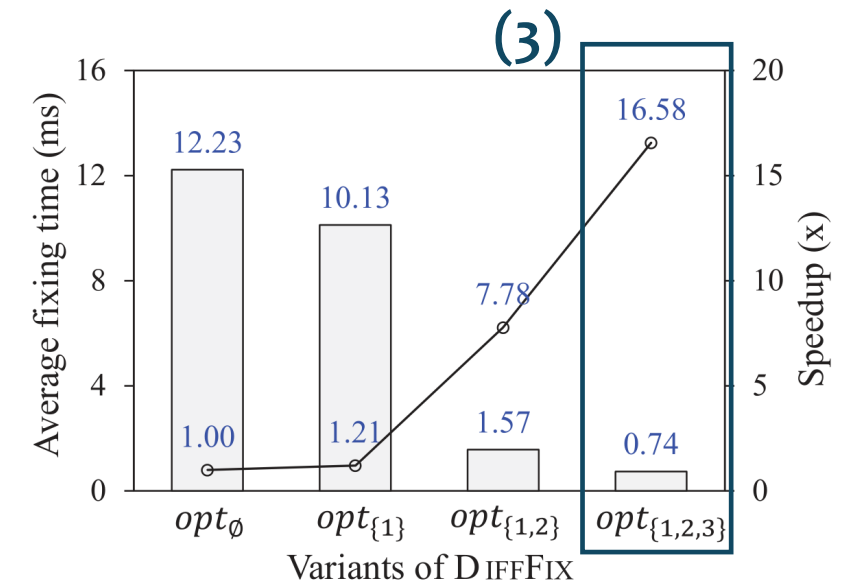
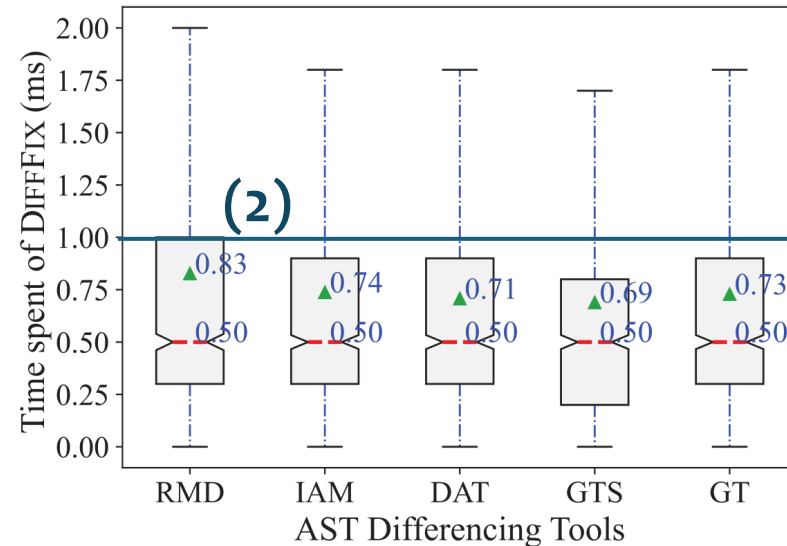
Answer to RQ1:

DIFFFIX effectively improved the accuracy of diffs generated by five techniques for real-world commits, demonstrating high precision and generalizability.

Efficiency (RQ2)

- (1) DIFFFIX has a **negligible overhead (<2%)** on four techniques, and take $\approx 1/4$ of GTS matching time
- (2) DIFFFIX processed most diffs in **less than 1 ms**, the maximum is 9.4-13.0 ms across techniques
- (3) Three optimizations improved DIFFFIX's efficiency, ultimately achieving a **16.58x speedup**

	T_{diff} (ms)
RMD	34,108.1
RMD+DIFFFIX	34,769.5 ($\uparrow 1.94\%$)
IAM	96,568.5
IAM+DIFFFIX	97,158.0 ($\uparrow 0.61\%$)
DAT	32,472.9
DAT+DIFFFIX	33,038.4 ($\uparrow 1.74\%$)
GTS	2,206.3
GTS+DIFFFIX	2,757.1 ($\uparrow 24.96\%$)
GT	370,413.4
GT+DIFFFIX	370,995.9 ($\uparrow 0.16\%$)



Answer to RQ2:

DIFFFIX demonstrated high efficiency with negligible time cost on most techniques. The optimizations significantly improved its efficiency.

Effect on Edit Script Size (RQ3)

- (1) DIFFFIX reduced ES size on four techniques (↓3~11 actions per diff on average)
- (2) DIFFFIX slightly increased ES size on DAT (↑3 actions per diff on average)

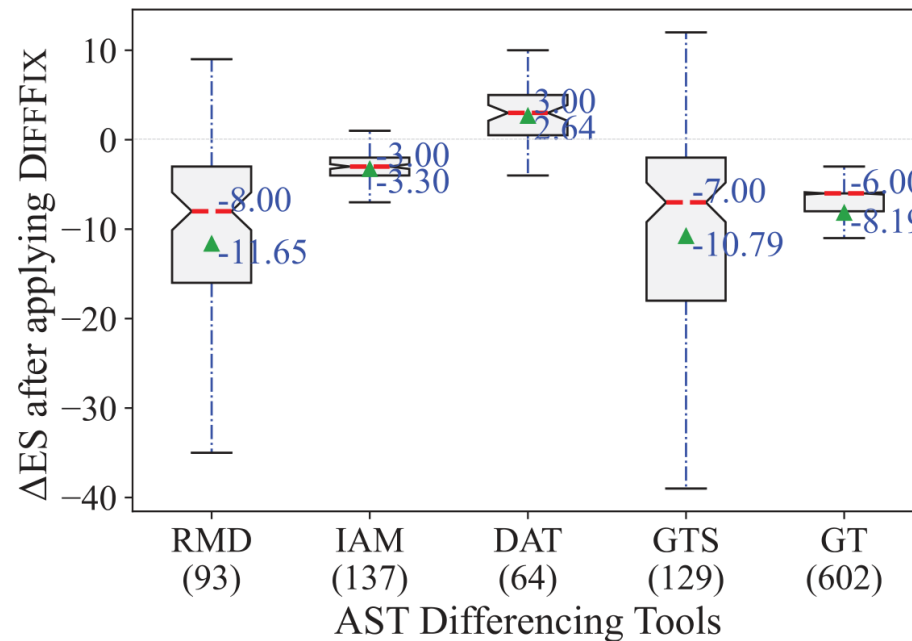
fixing missing mappings



reduce unnecessary deletions&additions



↓ ES Size



fixing arbitrary mappings



cancel confusing moves&updates



↑ ES Size

Answer to RQ3:

DIFFFIX reduced ES size on most techniques by fixing inaccurate mappings.

Impact of Subprocesses (RQ4)

- (1) The warm-up phase fixed some inaccurate mappings in RMD and IAM related to identical subtrees
- (2) The contributions of fixing passes were widely distributed across techniques
- (3) The warm-up phase took up longer time than the whole fixing phase
- (4) All passes had execution times of the same order of magnitude, and MatchChildren (MC) took the most time.

(a) The number of diffs that subprocesses take effect on (#)

	Warm-up (1) Phase	(2) Fixing Passes								
		MU	FP	FC	FI	FN	MI	MN	MP	MC
RMD	8	<u>45</u>	4	7	7	11	<u>22</u>	16	<u>20</u>	12
IAM	30	4	7	7	3	20	<u>38</u>	1	<u>26</u>	<u>75</u>
DAT	0	4	10	<u>56</u>	<u>33</u>	<u>23</u>	1	2	4	10
GTS	0	<u>41</u>	1	<u>71</u>	4	26	25	<u>30</u>	<u>29</u>	7
GT	0	26	4	<u>70</u>	23	<u>27</u>	3	12	20	<u>681</u>

(b) Total time spent of subprocesses (ms)

	Warm-up (3) Phase	Fixing Passes								(4)
		MU	FP	FC	FI	FN	MI	MN	MP	
RMD	377.6	13.5	7.9	13.6	15.5	16.8	<u>34.9</u>	10.1	<u>18.1</u>	<u>53.1</u>
IAM	386.8	13.9	9.9	16.2	18.4	19.0	37.0	12.7	<u>24.7</u>	<u>58.4</u>
DAT	371.7	8.5	9.9	14.9	20.3	22.5	<u>34.0</u>	12.3	25.0	62.8
GTS	365.9	13.7	7.7	15.6	18.1	<u>20.0</u>	<u>33.9</u>	13.5	19.5	<u>60.1</u>
GT	384.4	17.3	9.0	16.3	18.0	20.1	<u>32.6</u>	14.5	<u>23.4</u>	<u>62.4</u>

Answer to RQ4:

All fixing passes have contributions that vary across techniques.
The warm-up phase bore the main time cost among subprocesses.

Discussion

■ Key Features of DIFFFIX

- An **effective** and **efficient** AST diff fixing approach
- Post-processing, **incremental**, **context**(↓FN)+**type**(↓FP), optimized

■ Main Implications

- Inaccurate mappings exhibit certain **frequent and generalizable patterns**
- Context and type information guide establishing accurate node mappings
- Coexisting inaccurate mappings can be incrementally resolved

■ Future Work

- Tackle **subtle inaccurate mappings**
 - mainly related to variables/method invocations
 - introduce lightweight **semantic analysis** into DiffFix
- Establish a continuously updated public **AST diff database**
 - report inaccurate diffs / provide verified accurate diffs
- Investigate **the impact of ASTDiff accuracy on downstream tasks**
 - conduct systematic studies

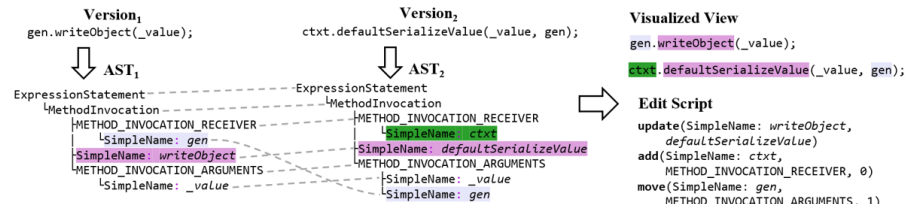
Summary

Overview of AST Differencing (ASTDiff)

■ Process

- (1) parse two versions of programs into ASTs
- (2) compare ASTs to get the diff (node mappings)
- (3) generate edit script / visualize syntactic change

A fundamental approach to capture structural and syntactic changes of program elements



■ Techniques

ChangeDistiller, GumTree, MTDiff, IJM, iASTMapper, RM-ASTDiff, etc.

■ Applications

automated program repair, change pattern mining, refactoring detection, etc.

2

Accuracy Issue of ASTDiff Techniques

■ Can the generated diffs accurately reflect the code changes?

■ Limitations of existing ASTDiff techniques

- Unsound node matching heuristic
 - Insufficient use of program information
 - Trade-off between accuracy and efficiency
 - Mutual interference of inaccuracies
- Struggle to generate accurate diffs, resulting in 20%-40% inaccuracies^[1,2]

Motivation:

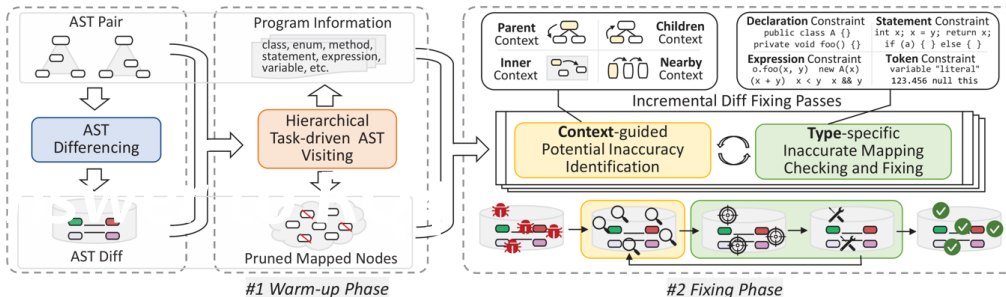
Since inaccurate mappings are difficult to prevent, can we identify and fix them in the generated diff to improve the accuracy?

[1] Y. Fan, X. Xia, D. Lo, et al. A differential testing approach for evaluating abstract syntax tree mapping algorithms. ICSE, 2021.

[2] P. Alikhanifard and N. Tsantalis. A novel refactoring and semantic aware abstract syntax tree differencing tool and a benchmark for evaluating the accuracy of diff tools. TOSEM, 2025.

3

Our Approach: DIFFIX



■ **Input:** A pair of ASTs and their diff generated by an ASTDiff technique

■ **Output:** A refined diff with inaccurate mappings fixed

#1 Warm-up Phase performs initializations and optimizations

#2 Fixing Phase performs identification and fixing of inaccuracies

8

Discussion

■ Key Features of DIFFIX

- An **effective** and **efficient** AST diff fixing approach
- Post-processing, **incremental**, **context**(↓FN)+**type**(↓FP), optimized

■ Main Implications

- Inaccurate mappings exhibit certain **frequent** and **generalizable** patterns
- Context and type information guide establishing accurate node mappings
- Coexisting inaccurate mappings can be incrementally resolved

■ Future Work

- Tackle **subtle inaccurate mappings**
 - mainly related to variables/method invocations
 - introduce lightweight **semantic analysis** into DiffFix
- Establish a continuously updated public **AST diff database**
 - report inaccurate diffs / provide verified accurate diffs
- Investigate **the impact of ASTDiff accuracy on downstream tasks**
 - conduct systematic studies

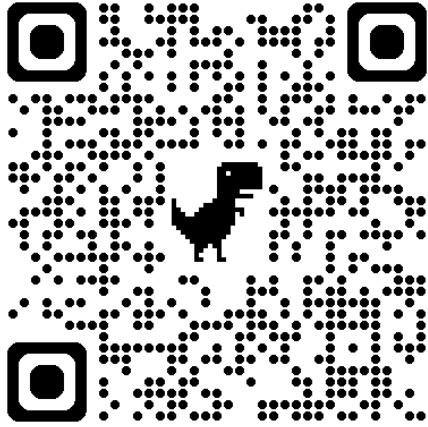
20

21

The 40th IEEE/ACM International Conference
on Automated Software Engineering (ASE 2025)

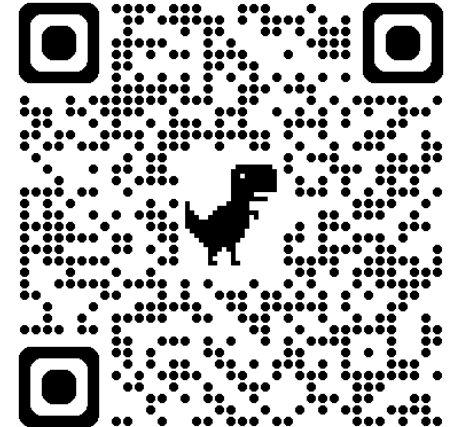


北京科技大学
University of Science and Technology Beijing



Paper

Thank You!



Code

Guofeng Zeng

d202110394@xs.ustb.edu.cn

Chang-ai Sun

casun@ustb.edu.cn

Kai Gao

kai.gao@ustb.edu.cn

Huai Liu

hliu@swin.edu.au

Seoul, November 19, 2025